

Phoenix Flores

CIS 344

Yanilda Ramos

7 May 2026

Grand Line Realty Portal Project

Project Purpose:

The main purpose of this project is to demonstrate understanding of creating sessions for users and implementing account creation/management. Moreover, demonstrating being able to use PHP and SQL to create a working website with a database for storing data about users, properties, transactions and inquiries users may have about properties.

Database Design:

The design for the database has 5 tables:

Users (userID(PK), userName, contactInfo, passwordHash, userType)

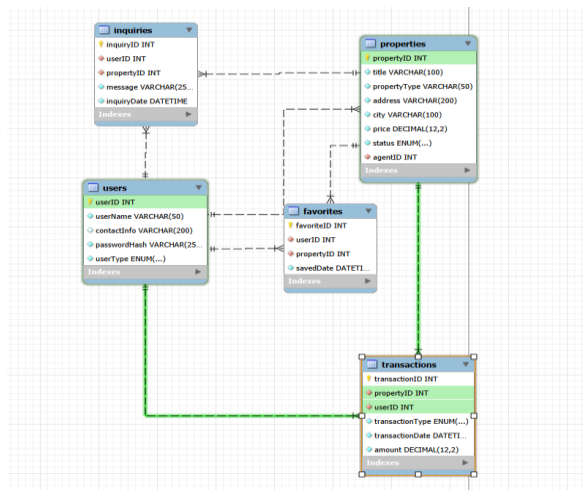
Properties (propertyID(PK), title, propertyType, address, city, price, status, agentID(FK))

Inquiries (inquiryID(PK), userID(FK), propertyID(FK), message, inquiryDate)

Transactions (transactionID(PK), propertyID(FK), userID(FK), transactionType, transactionDate, amount)

Favorites (favoriteID(PK), userID(FK), propertyID(FK), savedDate)

EER Diagram (From MYSQL Workbench):



PHP functionality:

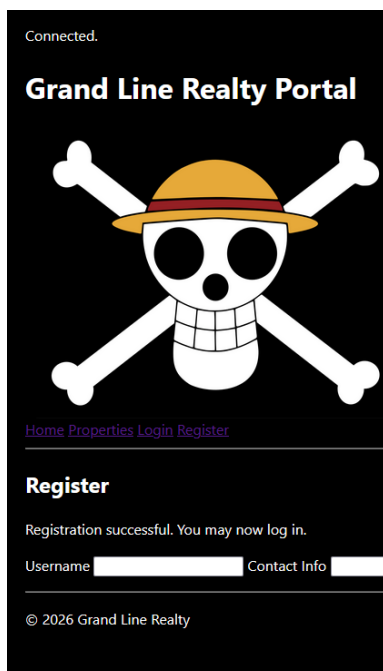
Section 1.

1.1.: Session handling

In the header file for site (which is included in each page) requires the file config.php once. This allows the site to initialize the definitions for the Database, such as the hostname, db name, username, and password. For the purposes of simplifying the file for the means of this project, the config file contains simple values as well as a check that verifies whether a session exists and starts one if it doesn't. This is important because if there is no session, there is no way to retrieve the data input from the user or establish a connection to be able to access the SQL database, which is why we put it in the config file, which gets called by the header, which is included in every page on the site.

1.2-3.: Register & Login handling using password hashing

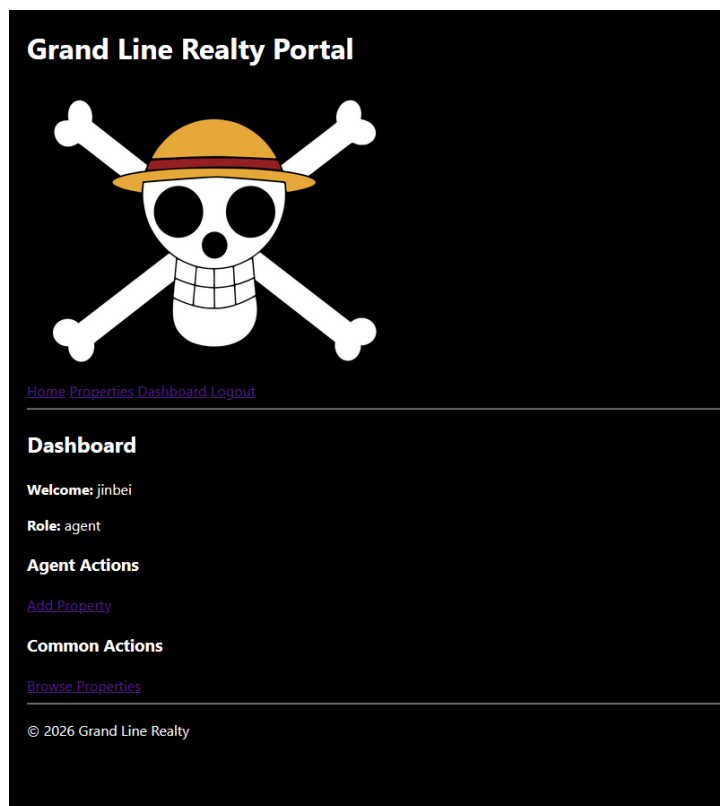
As for registering and logging into the site, we have register.php and login.php. register.php ensures a session is started and creates an object from realestatedatabase.php so we can call the getUserByUsername() function. Then, it extracts the user's input via the POST method to run an if-else statement given the user's input values for all fields. This if statement sets the password hash for the user using `password_hash(password, PASSWORD_DEFAULT)` and uses `addUser()` with the given info and newly created hash to enter a log into the database. Registration once successful should look something like this:



The next section will show what it is looked like when a user successfully logs in. Once a user takes their username and password to the login screen, the login.php file behaves similarly to the register file, ensuring a session is started, retrieving the username and password via the POST method, but this time it checks the password using the password_verify() function to check whether the hash of the provided password matches with the hash of the one we have stored in the database. If they match, the session matches the user with their account and redirects them to dashboard.php, which we will also look at in the next section.

Section 2. Role-based access control:

Since we want different permissions for users depending on who they are in relation to the database, different types of users will have access to different functions of the site. In the following screenshots, you will see what the user sees upon successful login. Given that there are different user types, their screens will be different. Provided is an example of a buyer and an agent logged in along with their hyperlinks pointing them to what they can access, as well as common options like viewing the properties listed.



Grand Line Realty Portal



[Home](#) [Properties](#) [Dashboard](#) [Logout](#)

Dashboard

Welcome: Nico Robin

Role: buyer

Common Actions

[Browse Properties](#)

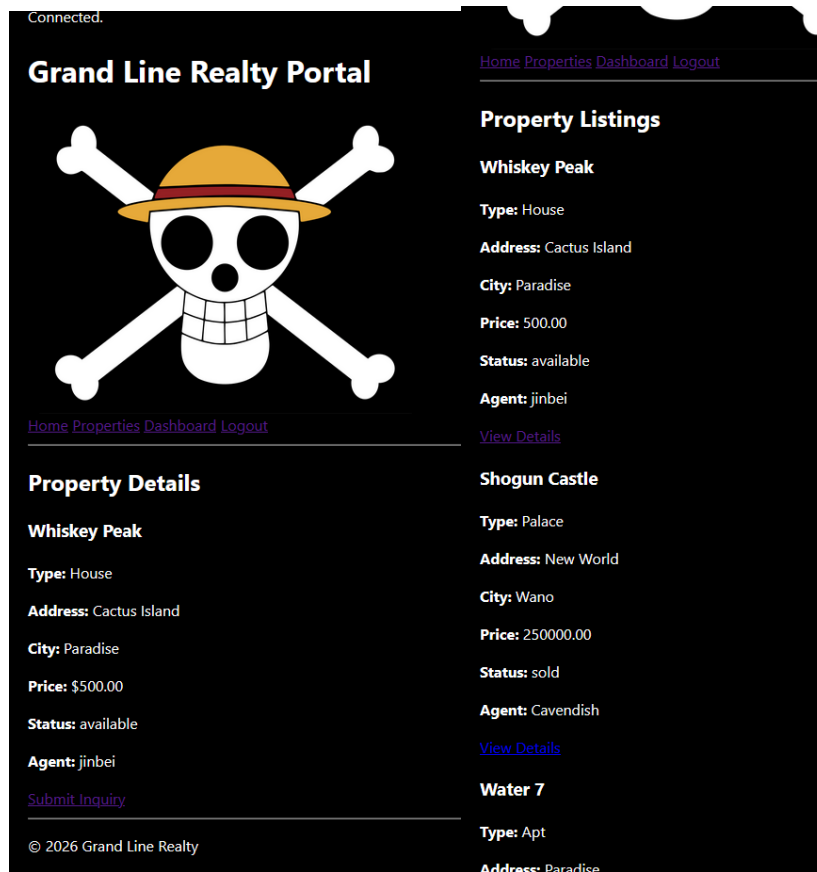
© 2026 Grand Line Realty

As we can see, since Jinbei is an agent, he can add a property to the list, which looks like this:

Add Property

Title Property Type Address City Price Status

However, since Nico Robin is only a buyer, she can only view properties and make inquiries on them, like listed in the following screenshots.



Section 3. Challenges and solutions:

The biggest problem I faced was multiple errors involving MySQLI. I was not as familiar with PDO objects and as such wanted to simplify the database connection process with MySQLI, which I used for the group project and seemed straightforward. I ended up having to change up a few methods that were provided to be able to work with the new type of connections but I ended up learning a lot about how function logic works when using SQL queries, and was thankfully able to lean on a bit of the knowledge I gained during the group project. In the end, I updated the database.php files along with some of the function calls in RealEstateDatabase.php when I added code to make the other ones work.

```

}

public function addProperty(string $title, string $propertyType, string $address, string $city, float $price, string $status, int $agentId): bool {
    $sql = "INSERT INTO properties (title, propertyType, address, city, price, status, agentID)
        VALUES (?, ?, ?, ?, ?, ?, ?)";
    $stmt = $this->conn->prepare($sql);
    $stmt->bind_param("ssssdsi", $title, $propertyType, $address, $city, $price, $status, $agentId);
    $stmt->execute();
    $result = $stmt->get_result();
    if (!$result) {
        return False;
    }
    return True;
}

public function getAllProperties(): array {
    // TODO: Optionally replace this with the PropertyListView.
    $sql = "SELECT p.*, u.userName AS agentName
        FROM properties p
        JOIN users u ON p.agentID = u.userID
        ORDER BY p.propertyID DESC";
    $result = mysqli_query($this->conn, $sql);
    if (!$result) {
        return [];
    }
    return $result->fetch_all(MYSQLI_ASSOC);
}

public function getPropertyById(int $propertyId) {
    $sql = "SELECT p.*, u.userName AS agentName
        FROM properties p
        JOIN users u ON p.agentID = u.userID
        WHERE p.propertyID = ?";
    $stmt = $this->conn->prepare($sql);
    $stmt->bind_param('i', $propertyId);
    $stmt->execute();
    $result = $stmt->get_result();
    return $result->fetch_assoc();
}

```

As you can see, I had to return the arrays from `getPropertyById` and `getAllProperties` to fetch an associative array (i.e. an array with keys for column names) that way the `properties.php` file can properly read and display the property information from that returned array.

Final Thoughts:

I had a fun time working with this project, even if I had an error code I was not familiar with that left me stuck for a while. (thank you, associative arrays) I am glad I was able to use `css` and `html` in tandem with what I learned about `PHP` to make this website function and somewhat stylized to me (though I could always do more for the style). It feels like connecting a bridge or maybe more like putting the last foundational pieces into an unfinished building. Regardless, this project taught me a lot and helped me understand where I'm lacking (syntax, data manipulation and bridging languages together).